

SCUOLA UNIVERSITARIA PROFESSIONALE DELLA SVIZZERA ITALIANA

Dipartimento Tecnologie Innovative (DTI)

Bachelor of Science in Ingegneria Informatica

DOCUMENTAZIONE TECNICA DI PROGETTO

JUST 21

Sviluppo di un'applicazione multi-modulo con interfaccia grafica asincrona, persistenza e integrazione nativa

Studenti: Felice Rossetti & Mattia Alongi

Docenza: Corso di Ingegneria del Software 1 – Prof. Amos Brocco & Prof. Edoardo Terzi

Anno Accademico: 2025-2026

INDICE GENERALE

1. INTRODUZIONE.....	3
1.1 Obiettivo del progetto	3
1.2 Contesto accademico e metodologico.....	3
1.3 Tecnologie usate e motivazioni	3
2. ANALISI E PROGETTAZIONE.....	4
2.1 Requisiti funzionali e non funzionali	4
2.2 Architettura multi-modulo esagonale	4
2.3 Scelte progettuali e Design Pattern	4
2.4 Diagramma delle Classi del Modello del Dominio.....	5
3. IMPLEMENTAZIONE.....	6
3.1 Struttura fisica del progetto (Maven)	7
3.2 Componenti principali del Backend	7
3.3 Componenti principali del Frontend.....	7
3.4 Funzionalità core e diagramma di stato.....	8
4. VERIFICA DEL FUNZIONAMENTO.....	10
4.1 Strategia di testing automatico (JUnit 5).....	10
4.2 Casi di test provati e riproducibilità.....	10
4.3 Test manuali ed esperienza utente (UX)	10
4.4 Ingegneria e gestione degli errori.....	10
5. PROBLEMI E SOLUZIONI	11
5.1 Difficoltà incontrate e bug risolti.....	11
5.2 Compromessi ingegneristici e debito tecnico.....	11
6. CONCLUSIONI E SVILUPPI FUTURI	12
6.1 Risultati ottenuti	12
6.2 Miglioramenti possibili ed evoluzione cloud.....	12

INDICE DELLE FIGURE:

Figura 1 - Blackjack - Classi Core (Gameplay + AI + persistenza)	5
Figura 2 - Diagramma di stato - enum GameState (Blackjack).....	8

1. INTRODUZIONE

1.1 Obiettivo del progetto

Il progetto consiste nello sviluppo di un'applicazione desktop cross-platform per il gioco del Blackjack. L'architettura del sistema separa la logica di gioco dall'interfaccia grafica e include funzionalità quali multiplayer locale con bot, persistenza delle partite tramite file JSON, integrazione di codice nativo tramite JNI e gestione audio tramite JavaFX.

1.2 Contesto accademico e metodologico

Il progetto è stato sviluppato nell'ambito del corso di Ingegneria del Software presso il Dipartimento Tecnologie Innovative (DTI) della SUPSI. Durante lo sviluppo sono stati applicati principi di modularità e separazione delle responsabilità, con particolare attenzione al basso accoppiamento tra i componenti e alla manutenibilità del codice. Sono inoltre stati utilizzati alcuni design pattern per semplificare la gestione della logica di gioco, della persistenza e dei componenti audio.

1.3 Tecnologie usate e motivazioni

Le tecnologie utilizzate sono state scelte per garantire portabilità, modularità e facilità di manutenzione del progetto:

- **Java JDK 21 (LTS):** linguaggio principale del progetto, utilizzato anche per funzionalità moderne come Records e pattern matching.
- **Apache Maven 3.9+:** utilizzato per la gestione del progetto multi-modulo e delle dipendenze.
- **JavaFX 21.0.4:** framework grafico utilizzato per l'interfaccia utente, i file FXML, i fogli CSS e le animazioni.
- **Jackson Framework 2.18.2:** libreria utilizzata per il salvataggio e caricamento delle partite in formato JSON.
- **GCC / MinGW:** compilatore C utilizzato per creare la libreria nativa integrata tramite JNI.

2. ANALISI E PROGETTAZIONE

2.1 Requisiti funzionali e non funzionali

Il sistema risponde a un preciso perimetro di requisiti ingegnerizzati per simulare un contesto operativo reale. Per maggiori dettagli consultare la sezione “Issues” presente sulla repository Github del progetto

2.2 Architettura multi-modulo esagonale

Il progetto separa la logica di gioco dai componenti grafici e di persistenza. Al centro risiede il Dominio di business, isolato e protetto all'interno del modulo Maven backend. Le interfacce grafiche, i driver multimediali e i file system si collocano all'esterno dell'esagono e agiscono come Adattatori (Adapters) infrastrutturali, dipendendo dal core senza introdurre dipendenze dirette. Questo garantisce che modifiche ai dettagli periferici (es. aggiornamento del look & feel visivo tramite fogli CSS) non abbiano alcuna ripercussione sulle regole logiche o matematiche del gioco del Blackjack.

2.3 Scelte progettuali e Design Pattern

La modularità e la manutenibilità del progetto sono state migliorate tramite l'utilizzo di alcuni design pattern:

- **Strategy Pattern (AI dei Bot):** L'interfaccia **PlayerStrategy** definisce il comportamento dei giocatori controllati dalla CPU. La classe **DealerMimicStrategy** implementa una strategia semplice basata sulle regole del dealer: il bot continua a pescare fino a 17 e non utilizza azioni come split o assicurazione. Questa soluzione permette di aggiungere facilmente nuove strategie senza modificare la classe **Player**.
- **Memento Pattern & DTO (Persistenza pulita):** La classe immutabile **GameSnapshot** viene utilizzata per salvare lo stato della partita, inclusi portafogli e cronologia dei round. Il salvataggio avviene nello stato **ROUND_OVER**, evitando modifiche dirette agli oggetti del dominio e semplificando la serializzazione tramite Jackson.
- **Asset Registry & Singleton (Gestione Audio):** La classe **SoundManager** centralizza il caricamento e la riproduzione degli effetti audio dell'applicazione. I file sonori vengono caricati all'avvio e memorizzati in una **EnumMap**, riducendo accessi ripetuti al disco durante il gioco.

2.4 Diagramma delle Classi del Modello del Dominio

Blackjack — Classi Core (gameplay + AI + persistenza)

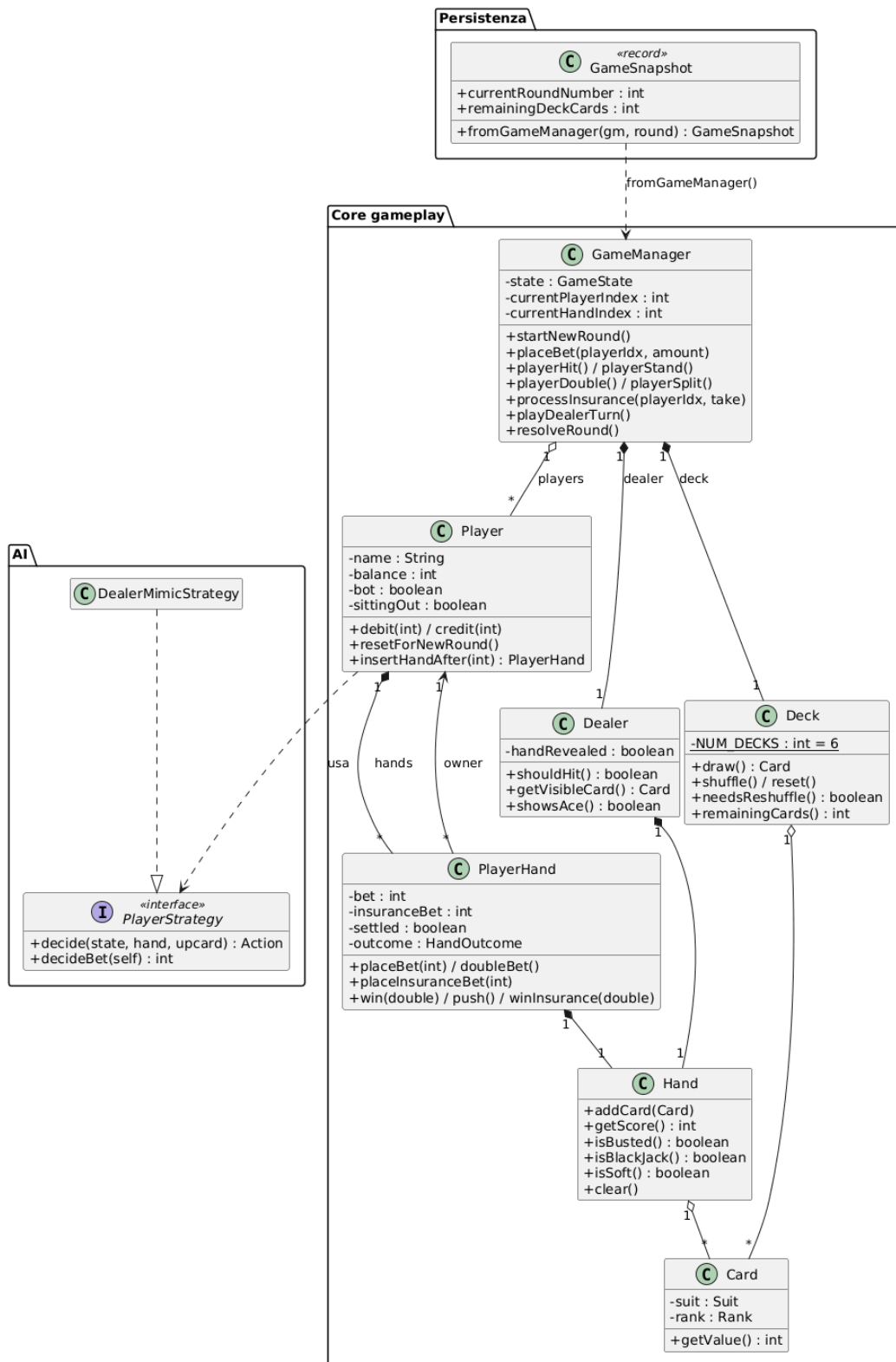


Figura 1 - Blackjack - Classi Core (Gameplay + AI + persistenza)

Spiegazione testuale del Diagramma delle Classi:

Il diagramma delle classi rappresenta le principali componenti del dominio del gioco del Blackjack e le relazioni tra di esse.

La classe centrale è GameManager, responsabile della gestione del flusso della partita, dei turni dei giocatori e delle interazioni con dealer, mazzo e mani di gioco.

Ogni Player può possedere una o più istanze di PlayerHand, permettendo la gestione della funzionalità di split. La classe PlayerHand contiene informazioni relative alla mano corrente, come puntata, assicurazione ed esito finale.

La classe Hand gestisce invece la collezione di carte e il calcolo del punteggio. Le singole carte sono rappresentate dalla classe Card, composta da seme (Suit) e valore (Rank).

Il comportamento dei bot è separato dal modello del giocatore tramite l'interfaccia PlayerStrategy, implementata dalla classe DealerMimicStrategy.

La persistenza dello stato della partita è rappresentata dalla classe GameSnapshot, utilizzata per salvare le informazioni necessarie al ripristino di una sessione.

3. IMPLEMENTAZIONE

3.1 Struttura fisica del progetto (Maven)

Il progetto utilizza una struttura Maven multi-modulo composta da un modulo **backend** e un modulo **frontend**:

- **Backend/src/main/java/.../game/**: contiene la logica del gioco, le classi del dominio, la gestione delle partite e i servizi di persistenza.
- **Frontend/src/main/java/.../controller/**: contiene i controller JavaFX, la gestione delle schermate e i componenti audio.
- **Frontend/src/main/resources/ui/**: contiene file FXML, fogli CSS e risorse grafiche dell'interfaccia.

3.2 Componenti principali del Backend

Il modulo backend contiene le principali classi della logica di gioco:

- **GameManager.java**: Classe principale che gestisce il flusso della partita e le azioni dei giocatori (hit, stand, split, double).
- **PersistenceService.java**: Gestisce il salvataggio e il caricamento delle partite in formato JSON tramite Jackson.
- **LicenseChecker.java**: Espone il metodo nativo JNI utilizzato per la verifica della licenza.

3.3 Componenti principali del Frontend

Il modulo frontend contiene i controller JavaFX e la gestione dell'interfaccia utente:

- **GameController.java**: Coordina l'interazione tra interfaccia grafica e backend, aggiornando dinamicamente la UI.
- **SoundManager.java**: Gestisce effetti sonori e musica di sottofondo tramite JavaFX MediaPlayer.
- **Navigation.java**: Gestisce il cambio delle schermate dell'applicazione.

3.4 Funzionalità core e diagramma di stato

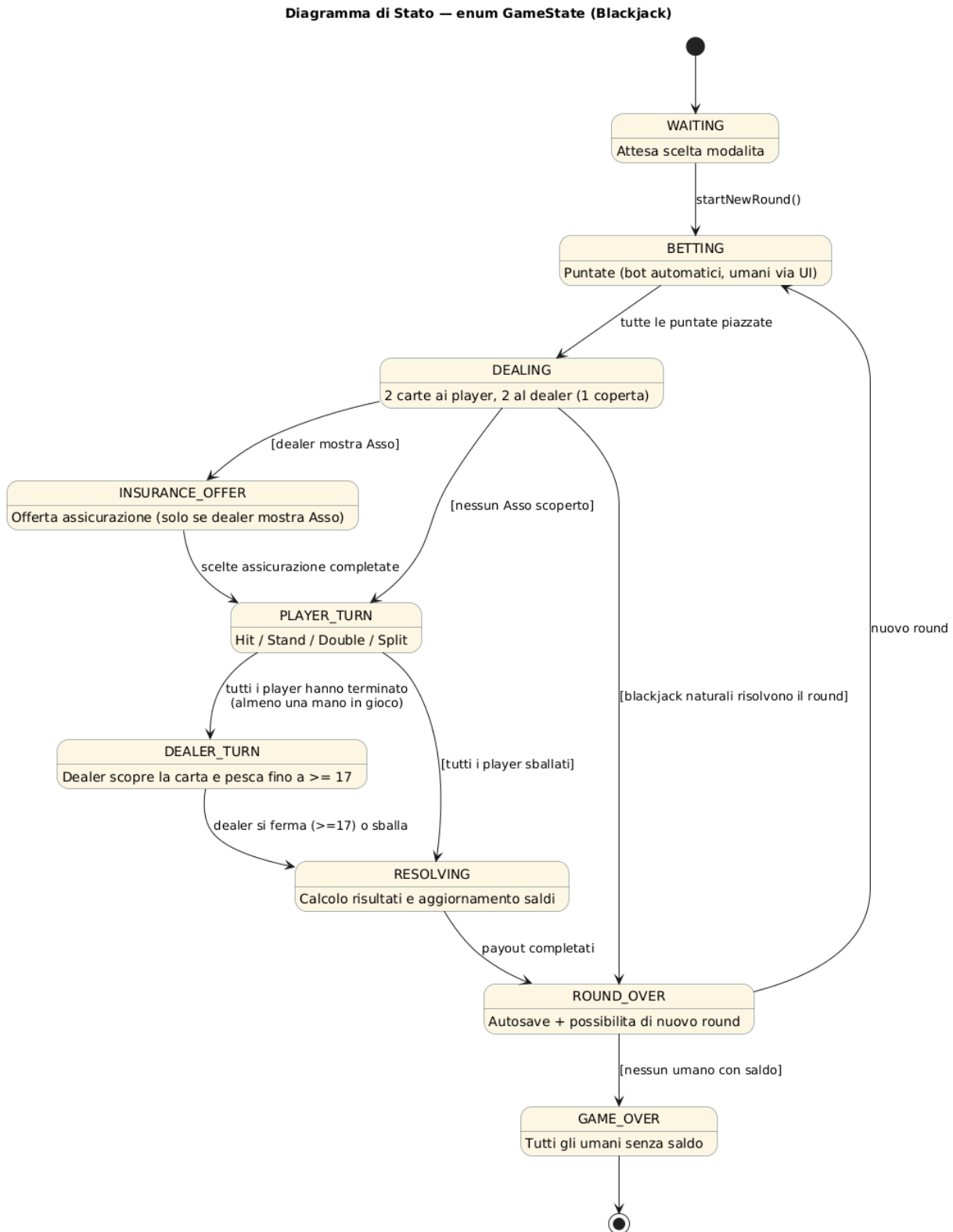


Figura 2 - Diagramma di stato - enum GameState (Blackjack)

Spiegazione testuale del Diagramma di Stato:

Il flusso della partita è gestito tramite una macchina a stati rappresentata dall'enum GameState.

La partita inizia nello stato **WAITING**, in cui il sistema attende la scelta della modalità di gioco. Successivamente si passa a **BETTING**, fase in cui i giocatori effettuano le proprie puntate; i bot puntano automaticamente una parte del loro saldo, mentre i giocatori umani utilizzano l'interfaccia grafica.

Dopo le scommesse, il sistema entra nello stato **DEALING**, distribuendo due carte ai giocatori e due al dealer (una coperta). Se il dealer mostra un Asso, viene attivato lo stato **INSURANCE_OFFER**, che consente ai giocatori di scegliere se effettuare l'assicurazione.

Terminate le verifiche iniziali sui Blackjack naturali, il gioco passa allo stato **PLAYER_TURN**, in cui ogni giocatore può eseguire azioni come **Hit**, **Stand**, **Double** e **Split**. I bot eseguono automaticamente le proprie mosse in base alla strategia associata.

Quando tutti i giocatori hanno terminato il turno, il sistema entra nello stato **DEALER_TURN**: il dealer rivela la carta coperta e pesca fino a raggiungere almeno 17 punti.

Successivamente viene eseguita la fase **RESOLVING**, durante la quale vengono calcolati i risultati delle mani e aggiornati i bilanci dei giocatori.

Il round termina nello stato **ROUND_OVER**, dove viene effettuato il salvataggio automatico della partita e viene resa disponibile la possibilità di iniziare un nuovo round.

Se tutti i giocatori umani esauriscono il proprio saldo, il sistema entra infine nello stato **GAME_OVER**.

4. VERIFICA DEL FUNZIONAMENTO

4.1 Strategia di testing automatico (JUnit 5)

La logica del Blackjack è stata verificata tramite test di unità e integrazione sviluppati con JUnit 5 nel modulo backend.

Uno dei principali problemi nel testing di un gioco di carte riguarda la casualità del mazzo, che rende difficile riprodurre gli stessi scenari durante l'esecuzione automatica dei test..

4.2 Casi di test provati e riproducibilità

Per rendere i test deterministici è stata creata una versione controllata del mazzo, chiamata **StackedDeck**, ottenuta estendendo la classe **Deck**. Questa implementazione utilizza una sequenza predefinita di carte, permettendo di riprodurre scenari specifici durante i test.

Tra i casi verificati:

- **testPlayerBlackjackPays3to2()**: Verifica il corretto pagamento di un Blackjack naturale e la conclusione immediata del round.
- **testInsurancePays2to1OnDealerBlackjack()**: Controlla il funzionamento della meccanica di assicurazione nel caso di Blackjack del dealer.
- **testSplitSecondHandAuto21()**: Verifica la gestione dello split e il corretto aggiornamento delle mani multiple del giocatore.

4.3 Test manuali ed esperienza utente (UX)

I test manuali sull'interfaccia grafica hanno verificato il corretto funzionamento delle animazioni, dei controlli utente e degli effetti audio.

In particolare, è stata controllata la sincronizzazione tra distribuzione delle carte, aggiornamento della UI e riproduzione degli effetti sonori. È stato inoltre verificato il corretto funzionamento del controllo del volume tramite MediaPlayer..

4.4 Ingegneria e gestione degli errori

L'applicazione gestisce diversi possibili errori tramite controlli e blocchi **try-catch**:

- **Verifica Licenze JNI**: Se la libreria nativa non viene caricata correttamente, l'eccezione `UnsatisfiedLinkError` viene intercettata e l'interfaccia mostra un messaggio di errore senza interrompere l'esecuzione del programma.
- **Gestione file JSON corrotti**: Durante il caricamento dei salvataggi, eventuali file JSON non validi vengono ignorati senza compromettere gli altri slot di salvataggio.

5. PROBLEMI E SOLUZIONI

5.1 Difficoltà incontrate e bug risolti

Durante lo sviluppo sono emersi alcuni problemi legati alla gestione della UI e della logica di gioco:

1. **Formattazione del campo licenza:** Nel ***LicenseController***, l'inserimento automatico dei trattini nel ***TextField*** causava errori durante la modifica del testo, soprattutto in fase di cancellazione, generando l'eccezione JavaFX "***start must be <= end***". Il problema è stato risolto eseguendo l'aggiornamento del testo tramite ***Platform.runLater()***, evitando modifiche concorrenti al contenuto del campo..
2. **Gestione dello split nella UI:** Nelle prime versioni dell'interfaccia, il risultato della partita veniva calcolato considerando solo la prima mano del giocatore (***hands.get(0)***). In caso di split, la seconda mano veniva quindi ignorata. La soluzione adottata è stata spostare completamente il calcolo degli esiti nel backend, lasciando al frontend il solo compito di visualizzare i risultati delle singole mani.

5.2 Compromessi ingegneristici e debito tecnico

Per rispettare i tempi di sviluppo, sono stati accettati alcuni compromessi progettuali:

- **Percorsi relativi per i salvataggi:** I file JSON vengono salvati tramite percorsi relativi al progetto (***Path.of("frontend", "saved", filename)***), soluzione adatta all'esecuzione in ambiente IDE. In una futura versione distribuita tramite JAR, i salvataggi verranno spostati nella cartella utente tramite ***System.getProperty("user.home")***.
- **Creazione dinamica delle Timeline JavaFX:** Le animazioni dei bot e del dealer creano nuove istanze di Timeline a ogni turno. Sebbene il Garbage Collector gestisca correttamente la memoria, una soluzione più efficiente prevederebbe il riutilizzo di una singola istanza configurabile.

6. CONCLUSIONI E SVILUPPI FUTURI

6.1 Risultati ottenuti

Il progetto **Just 21** si configura come un'applicazione desktop estremamente stabile, performante e rispondente a tutti i requisiti qualitativi imposti dalle moderne metodologie di sviluppo software. L'adozione della Clean Architecture ha permesso di proteggere il cuore logico dell'applicazione, isolandolo dalle dipendenze grafiche e multimediali [cite: 536-537, 1476]. La suite di test automatizzati JUnit garantisce l'assenza di regressioni logiche ad ogni build [cite: 647, 1194], mentre l'integrazione asincrona delle timeline grafiche offre un'esperienza utente reattiva, accattivante ed esente da micro-congelamenti del thread principale [cite: 805-810, 1208-1210].

6.2 Miglioramenti possibili ed evoluzione cloud

L'elevata modularità dei pacchetti e il totale disaccoppiamento del modulo backend aprono la strada a due importanti sviluppi evolutivi futuri su scala enterprise:

1. **Migrazione verso un'Architettura Client-Server Online:** Grazie all'assenza di import grafici nel modulo core, la classe GameManager e l'intero motore del Blackjack possono essere estratti ed integrati all'interno di un server web centrale sviluppato in Spring Boot o Quarkus, fungendo da orchestratore di stanze di gioco distribuite sul cloud. Il modulo frontend attuale (JavaFX) verrebbe rifattorizzato sostituendo le invocazioni dirette in memoria con un client di rete basato su connessioni bidirezionali asincrone stabili via **WebSockets** per lo scambio dei messaggi JSON in tempo reale, mantenendo intatte le regole di business del dominio.
2. **Integrazione di Persistenza Relazionale Cloud centralizzata:** Il servizio PersistenceService, attualmente vincolato alla scrittura di file JSON locali su disco, può essere esteso implementando un repository JPA interfacciato con un database relazionale enterprise cloud (es. PostgreSQL). Questa evoluzione consentirebbe di centralizzare la gestione dei profili utente e di alimentare una Leaderboard globale in tempo reale condivisa tra tutti i client connessi alla rete, trasformando il gioco locale in una piattaforma multiplayer competitiva.